

HW1 Report 112550151

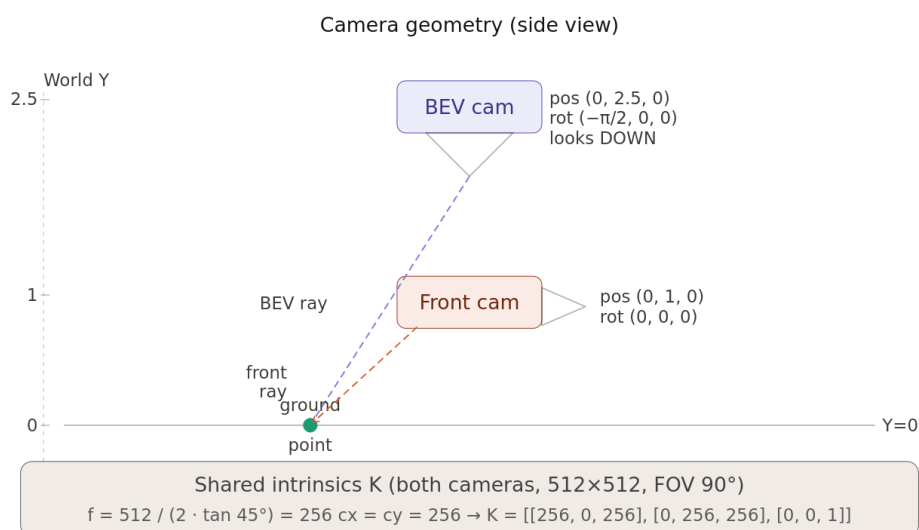
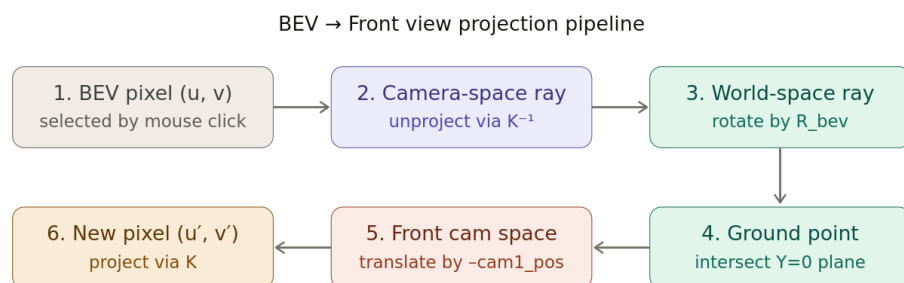
1. Implementation (70%)

a. Code

Detailed explanation of your implementation. For example, how you do the projection.

I first calculate the focal length, principal point, and rotation matrix and initialize the camera positions. Then I apply the projection to each pixel. I unproject the pixel with its intrinsic matrix to get the ray of the pixel in BEV and rotate the ray into world space. If the ray is parallel to the floor, that point is skipped. Then I calculate where the ray intersects the ground to get the point in world space and skip if it is behind the camera. Then I transform the point from world space into the front camera space and skip if it is behind this camera. Lastly, the points are added to the new_pixels array. The array is reshaped to fit cv2.fillPoly used in the show_image function.

Pipeline generated by Claude:



```

def top_to_front(self, theta=0, phi=0, gamma=0, dx=0, dy=0,
dz=0, fov=90):
    """
        Project the top view pixels to the front view pixels.
        :return: New pixels on perspective(front) view image
    """

    # calculate focal length
    fov_rad = np.deg2rad(fov)
    f = self.width / (2 * np.tan(fov_rad / 2))
    # principal point
    cx, cy = self.width / 2, self.height / 2

    # camera positions
    cam1_pos = np.array([0.0, 1.0, 0.0])
    cam2_pos = np.array([0.0, 2.5, 0.0])

    # BEV camera's rotation matrix
    angle = -(np.pi / 2)
    R_bev = np.array([
        [1, 0, 0],
        [0, np.cos(angle), -np.sin(angle)],
        [0, np.sin(angle), np.cos(angle)]
    ])

    new_pixels = []

    for p in points:
        u, v = p[0], p[1]

        # unproject BEV pixel to camera-space ray
        ray_cam = np.array([(u - cx) / f, (v - cy) / f, -1.0])
        # rotate ray into world space
        ray_world = R_bev @ ray_cam

        # skip if parallel
        if abs(ray_world[1]) < 1e-6:
            continue

        # where the ray from cam2_pos hits the ground
        t = -cam2_pos[1] / ray_world[1]
        # skip if behind camera

```

```

if t < 0:
    continue

world_pt = cam2_pos + t * ray_world

# transform world point into front camera space
p_cam1 = world_pt - cam1_pos

# skip if behind camera
if p_cam1[2] <= 0:
    continue

# project onto front image
u_front = int(round((f * p_cam1[0] / p_cam1[2]) + cx))
v_front = int(round((f * (-p_cam1[1]) / p_cam1[2]) + cy))

# If the projected points are out of bound, set to bound
u_front = max(0, min(u_front, self.width - 1))
v_front = max(0, min(v_front, self.height - 1))

new_pixels.append([u_front, v_front])

return np.array(new_pixels, dtype=np.int32).reshape((-1, 1,
2))

```

b. Result and Discussion

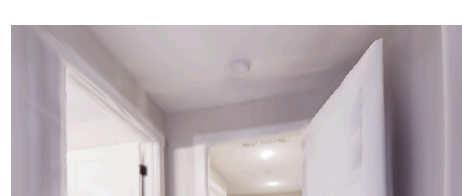
- i. Result of your projection (2 different pairs). Like the example result above.



bev1.png



front1.png



bev2.png

front2.png

ii. Anything you want to discuss.

Originally, I skipped points that were out of bounds in my for loop, which led to the `new_pixels` array to be empty sometimes and an error in `cv2.fillPoly`, as there isn't a polygon to be filled. However, this issue wasn't just happening with points that were out of bounds, even points that were supposed to be in the camera view were skipped too. I realised that my directions were incorrect, I had used `+1.0` for `Z` in `ray_cam`, which led to all the points being out of bound, after changing it to `-1.0`, this solved some points, but not all. The second issue was that I forgot to negate `Y` in `p_cam1[1]`, once I fixed this, all the points projected normally.

Additionally, I realised that if I changed the original camera position of camera 1 to `[0.0, 1.07, 0.0]`, the projection in the second set of images would be a little more accurate, but the projection in the first set would be off. This made me wonder that maybe the camera positions used in these sets of images were different, leading to this minor projection misalignment.

Also, since points out of bound are skipped, if 2 points are in bound and 2 are out of bound, the projection may be incorrect, however this isn't required to implement in this assignment so I ignored it.

iii. Any reference you take.

I looked at the references provided by TA to better understand this assignment.

1. http://vision.stanford.edu/teaching/cs131_fall1516/lectures/lecture8_camera_models_cs131.pdf
2. <https://staff.fnwi.uva.nl/r.vandenboomgaard/IPCV20162017/LectureNotes/CV/PinholeCamera/PinholeCamera.html>

2. Questions (30%)

- a. How do the intrinsic and extrinsic matrices contribute to the BEV-to-front-view projection?

The intrinsic matrix transforms the points from the 3D camera coordinate system to the 2D pixel coordinate system. It is used in this assignment to unproject a BEV pixel into a camera-space ray, then project the world point onto the front image.

On the other hand, the extrinsic matrix transforms the points from the world coordinate system to the camera coordinate system. It is used in this assignment to rotate the BEV camera ray into world space, and transform the world point into the front camera's coordinates.

- b. What factors may cause projection misalignment? List at least two factors and explain the reason.

If the camera positions are incorrect or inconsistent between images, the transformation may not work for a series of images as the incorrect coordinates would shift the projections and may lead to the projection being correct in one image but misaligned in another. Additionally, if the coordinate axes are different between cameras but aren't considered during transformation, it might project onto the wrong axes. For example in this assignment the BEV looks straight down, so its local Z axis aligns with world $-Y$, while the front camera looks forward, so its local Z axis aligns with world $+Z$.